

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



BLM4538 - iOS İLE MOBİL UYGULAMA GELİŞTİRME II
SplitSnap Projesi Final Raporu

Bora KOCABIYIK
21290270

Öğr. Gör. Enver BAĞCI

10/06/2026

github.com/brckfrc/splitsnap

ÖZET

Bu rapor, BLM4538 - iOS ile Mobil Uygulama Geliştirme II dersi kapsamında geliştirilen SplitSnap adlı iOS uygulamasının final teslimini belgelemektedir. SplitSnap, arkadaş grupları, ev arkadaşları ve gezi gruplarının ortak harcamalarını tek bir yapı altında toplayan, kimin ne kadar ödediğini ve grup içindeki borç-alacak durumunu otomatik olarak hesaplayan bir grup harcaması paylaşım uygulamasıdır. Uygulama React Native ve Expo çatısı üzerinde TypeScript ile geliştirilmiş; kimlik doğrulama, veritabanı, dosya saklama ve sunucusuz fonksiyonlar için Supabase altyapısı kullanılmıştır.

Proje yalnızca bir arayüz çalışması olmayıp; kimlik doğrulama, ilişkisel veritabanı tasarımı, satır düzeyinde güvenlik (Row Level Security), borç-alacak hesaplama algoritması, cihaz içi metin tanıma ile yapay zekâ destekli fiş okuma ve yerel veri saklama gibi uygulamalı pek çok teknik konuyu bir arada içermektedir. Dönem başında teslim edilen tanıtım raporunda “gelecek sürüm” olarak işaretlenen App Store yayını da tamamlanmış; uygulama gerçek bir ürün olarak Apple App Store’da yayımlanmıştır.

Geliştirme süreci on haftalık bir yol haritası üzerinden yürütülmüştür. İlk haftalarda proje altyapısı, kimlik doğrulama ve tema; orta haftalarda veritabanı tasarımı, grup ve harcama yönetimi ile borç-alacak hesaplaması; son haftalarda ise yerel veri saklama, fiş tarama ve App Store yayını tamamlanmıştır. Her aşamada işlevsel hedeflerin yanı sıra güvenlik, başarımlar ve kod kalitesi gözetilmiş; planlı bir denetimle tespit edilen açıklar ve darboğazlar giderilmiştir.

Rapor, tamamlanmış ürünü temel alan özellik odaklı bir yapıda kurgulanmıştır. İlk bölümlerde projenin amacı, kullanılan teknolojiler ve sistem mimarisi ile veritabanı tasarımı ve güvenlik modeli ele alınmakta; ardından kimlik doğrulama ve grup yönetimi, harcama yönetimi ve borç hesaplama, yapay zekâ destekli fiş tarama, performans optimizasyonları ve App Store yayın süreci ayrıntılandırılmaktadır. Son bölümde proje, dönem başı planıyla karşılaştırmalı olarak değerlendirilmekte; tamamlanan işler, karşılaşılan teknik zorluklar ve gelecek sürümlere bırakılan geliştirmeler özetlenmektedir.

İÇİNDEKİLER

1. GİRİŞ VE PROJE TANIMI	5
1.1. Problem ve Motivasyon	5
1.2. Projenin Amacı	5
1.3. Projenin Kapsamı ve Gerçekleşen Durum	5
2. KULLANILAN TEKNOLOJILER VE SİSTEM MIMARISI	7
2.1. Geliştirme Yığını	7
2.2. Sunucusuz Backend Yaklaşımı (Supabase)	7
2.3. Katmanlı Uygulama Mimarisi	8
3. VERİTABANI TASARIMI VE GÜVENLİK	9
3.1. Tasarım İlkeleri	9
3.2. İlişkisel Şema ve Tablolar	9
3.3. Otomatik Davranışlar: Tetikleyiciler ve Fonksiyonlar	9
3.4. Satır Düzeyinde Güvenlik (Row Level Security)	10
3.5. Güvenlik ve Tutarlılık Denetimi	10
4. KİMLİK DOĞRULAMA, GRUP VE ÜYELİK YÖNETİMİ	12
4.1. Kayıt, Giriş ve Oturum Yönetimi	12
4.2. Grup Oluşturma ve Üyelik	12
4.3. Davet Kodu ve Gruba Katılma	13
4.4. Tıklanabilir Davet Bağlantıları (Universal Link)	13
5. HARCAMA YÖNETİMİ VE BORÇ HESAPLAMA	14
5.1. Harcama Ekleme, Düzenleme ve Silme	14
5.2. Eşit ve Manuel Bölüşüm	14
5.3. Borç-Alacak Hesabı ve Ödüşme Optimizasyonu	14
5.4. Kullanıcı Deneyimi İyileştirmeleri	16
6. YAPAY ZEKÂ DESTEKLİ FİŞ TARAMA	17
6.1. Mimari Karar: Hibrit Model	17
6.2. Uçtan Uca İşleyiş	17
6.3. Dayanıklılık: Yerel Sezgisel Yedek	17
6.4. Para Birimi Tespiti ve Yabancı Döviz Uyarısı	18
6.5. Gizlilik ve Maliyet	18
6.6. Değerlendirilen Alternatif: GLM-OCR	18
7. YEREL VERİ SAKLAMA, PERFORMANS VE OPTİMİZASYON	20
7.1. MMKV ile Yerel Veri Saklama	20
7.2. Başarım Optimizasyonları	20
7.3. Paket Boyutu ve Derleme İyileştirmeleri	20
7.4. Form Doğrulama ve Profil Yönetimi	20
8. APP STORE YAYINI VE DAĞITIM	22
8.1. EAS ile Derleme ve Gönderim Hattı	22
8.2. App Store Connect Ayarları	22
8.3. Gizlilik Manifestosu ve Uyumluluk	23
8.4. Hesap Silme Akışı	23
8.5. Web Sitesi ve Davet Bağlantısı Altyapısı	24
8.6. Yayın Aşamasında Çözülen Sorunlar	24
9. SONUÇ VE DEĞERLENDİRME	26
9.1. Plana Göre Gerçekleşme	26
9.2. Teknik Kazanımlar	26

9.3. Karşılaşılan Zorluklar	26
9.4. Gelecek Geliştirmeler	26
KAYNAKÇA	27
Çatı, Dil ve İstemci Kütüphaneleri	27
Backend, Veritabanı ve Yapay Zekâ	27
Platform ve Dağıtım	27
Proje Çıktıları	27

1. GİRİŞ VE PROJE TANIMI

1.1. Problem ve Motivasyon

Birden çok kişinin birlikte yaptığı yemek, market, gezi veya ortak yaşam harcamalarında kimin ne kadar ödediği ve sonunda kimin kime ne kadar borçlu olduğu kolayca karışmaktadır. Harcamaların akılda ya da dağınık notlarda tutulması; eksik kayıt, hatalı bölüşüm ve gruplar arası anlaşmazlık gibi sorunlara yol açar. SplitSnap, bu günlük problemi mobil ortamda düzenli, hızlı ve anlaşılır bir biçimde takip edilebilir hâle getirmek amacıyla geliştirilmiştir.

Uygulamanın temel fikri basittir: kullanıcılar bir grup oluşturur ya da davet koduyla mevcut bir gruba katılır, yaptıkları harcamaları sisteme ekler, harcamayı kimin ödediğini ve kimler arasında paylaşılacağını belirtir. Sistem ise grup içindeki net borç-alacak durumunu ve en az sayıda işlemle herkesin nasıl ödeyeceğini otomatik olarak hesaplar.

1.2. Projenin Amacı

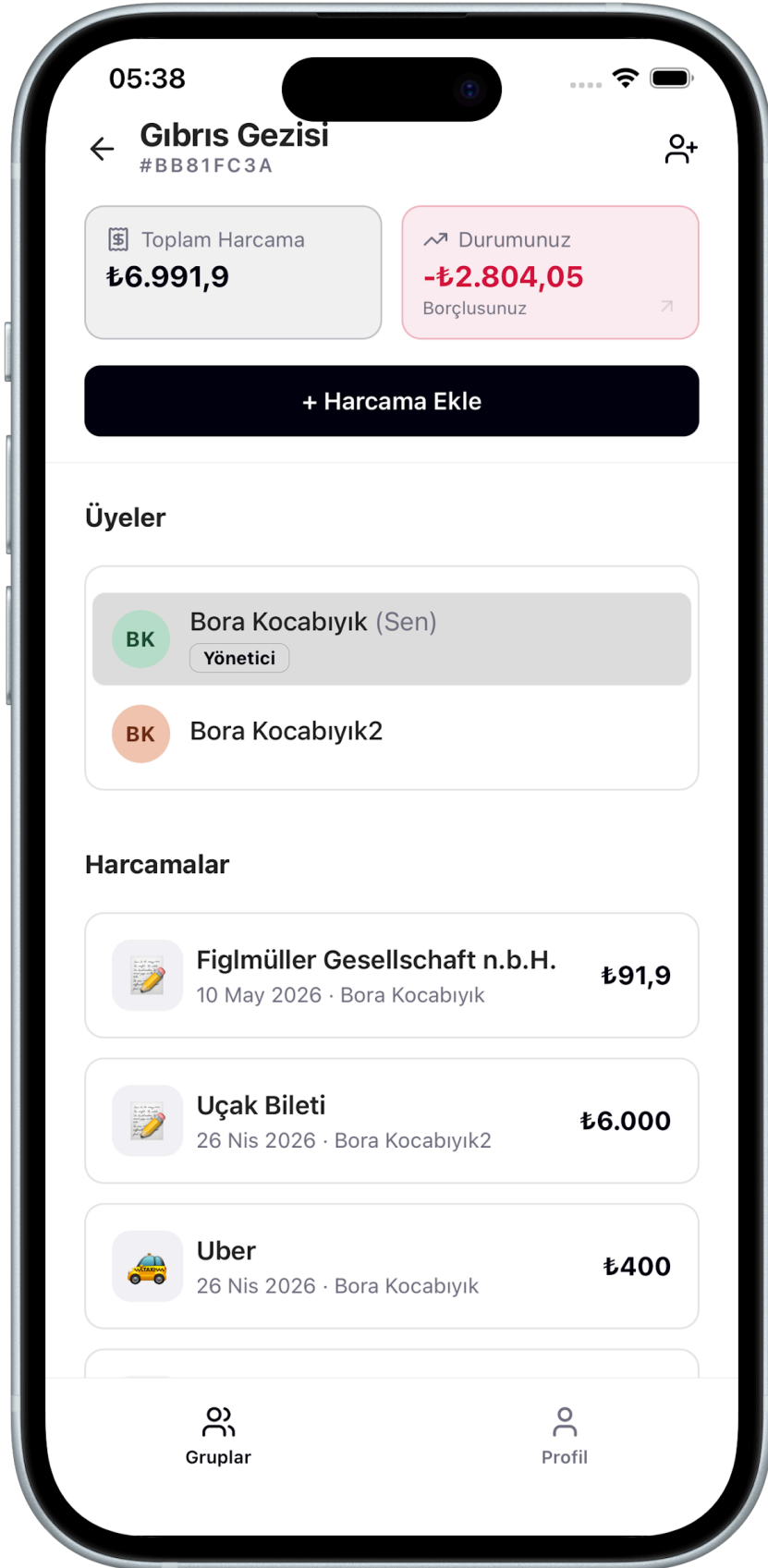
Projenin işlevsel amacı, ortak ödeme ve masraf paylaşımı sürecini tek bir sistem üzerinden takip edilebilir kılmaktır. Teknik amacı ise, modern bir mobil uygulamanın uçtan uca geliştirilmesinde rol alan katmanların tümünde uygulamalı deneyim kazanmaktır. Bu kapsamda proje aşağıdaki teknik hedefleri kapsamaktadır:

- React Native ve Expo ile iOS odaklı mobil uygulama geliştirme.
- TypeScript ile tip güvenli veri modeli ve uygulama yapısı kurma.
- Bir bulut altyapısı (Supabase) üzerinde kimlik doğrulama ve oturum yönetimi.
- Grup, üyelik, harcama ve pay ilişkilerini ilişkisel veritabanında modelleme.
- Satır düzeyinde güvenlik (RLS) politikalarıyla veri erişimini sınırlama.
- Borç-alacak hesaplama mantığını ve ödeme optimizasyonunu kodlama.
- Cihaz içi metin tanıma ve yapay zekâ ile fiş bilgisi otomatik çıkarma.
- Yerel veri saklama ile açılış performansını ve kullanıcı deneyimini iyileştirme.
- Uygulamayı gerçek bir ürün olarak App Store sürecinden geçirip yayımlama.

1.3. Projenin Kapsamı ve Gerçekleşen Durum

Dönem başındaki tanıtım raporunda çalışan ve sunulabilir bir iOS uygulaması hedeflenmiş; çekirdek sistem önceliklendirilmişti. Final teslimi itibarıyla planlanan tüm temel özellikler tamamlanmış, ayrıca plan dışı pek çok geliştirme eklenmiştir. Tamamlanan çekirdek özellikler şunlardır: kullanıcı kaydı ve girişi, oturum yönetimi, grup oluşturma ve davet koduyla gruba katılma, üye görüntüleme, harcama ekleme/düzenleme/silme, ödeyen ve katılımcı seçimi, eşit ve manuel bölüşüm, grup içi borç-alacak hesaplama, ödeme özeti, fiş görseli ekleme ve cihaz içi OCR ile temel alanların otomatik doldurulması, yerel veri saklama desteği.

Tanıtım raporunda açıkça “sonraki aşama / kapsam dışı” olarak belirtilen App Store yayın süreci de bu dönemde gerçekleştirilmiş ve uygulama Apple App Store’da yayımlanmıştır. Bu nedenle final teslimi, başlangıçta belirlenen hedefin ötesine geçmiştir. Fişten ürün bazlı tam ayırtırma, çoklu para birimi ve Android desteği gibi ileri özellikler ise bilinçli olarak gelecek sürümlere bırakılmıştır (Bölüm 9).



Şekil 1. SplitSnap grup ve harcama ekranı — uygulamanın temel kullanım görünümü.

2. KULLANILAN TEKNOLOJILER VE SİSTEM MIMARISI

2.1. Geliştirme Yığını

SplitSnap, tek bir kod tabanından iOS için derlenen bir React Native uygulamasıdır. Geliştirme ve dağıtım süreçlerini hızlandırmak için React Native'in üzerine kurulu Expo çatısı ve Expo Router tabanlı, dosya temelli yönlendirme yapısı tercih edilmiştir. Kod tarafında tip güvenliği ve sürdürülebilirlik için TypeScript kullanılmıştır. Proje dönem boyunca güncel tutulmuş; final aşamasında Expo SDK 56 (React Native 0.85) sürümüne yükseltilerek iOS 26.5 uyumluluğu sağlanmıştır.

Arayüz katmanında, tutarlı bir tasarım dili ve tema desteği için Tamagui bileşen kütüphanesi; uygulama genelinde durum yönetimi için Zustand; yüksek hızlı yerel veri saklama için MMKV kullanılmıştır. Tasarımlar önce Figma üzerinde hazırlanmış, ardından React Native bileşenlerine dönüştürülmüştür. Sürüm kontrolü Git ve GitHub ile yürütülmüştür.

Çizelge 1. SplitSnap teknoloji yığını ve katman sorumlulukları.

Katman	Teknoloji	Görev
Uygulama çatısı	React Native + Expo (SDK 56)	iOS derlemesi, yönlendirme, native modül erişimi
Dil	TypeScript	Tip güvenli model ve uygulama yapısı
Arayüz	Tamagui + Inter font	Tema, bileşen kütüphanesi, Sheet modalları
Durum yönetimi	Zustand	Grup/harcama/pay/ödeşme durumları, seçiciler
Yerel saklama	MMKV	Senkron, hızlı kalıcı önbellek (persist)
Backend	Supabase	Auth, PostgreSQL, Storage, Edge Functions, Realtime
Yapay zekâ	OpenAI gpt-4o-mini	Fiş metninden yapılandırılmış alan çıkarımı
Dağıtım	EAS Build + Submit	Bulutta derleme ve App Store'a gönderim

2.2. Sunucusuz Backend Yaklaşımı (Supabase)

Projede özel bir uygulama sunucusu yazılmamış; tüm backend ihtiyaçları Supabase üzerinden karşılanmıştır. Bu tercih, sınırlı dönem süresini sıfırdan sunucu geliştirmeye ayırmadan gerçek kullanıcı hesabı, ilişkisel veri ve güvenlik kurmaya olanak tanımıştır. Supabase'in kullanılan bileşenleri şunlardır:

- Supabase Auth — kullanıcı kayıt, giriş ve oturum yönetimi.
- PostgreSQL veritabanı — kullanıcı, grup, üyelik, harcama, pay ve ödeşme verileri.
- Storage — fiş görsellerinin özel (private) bir kovada saklanması.
- Edge Functions — fiş okuma ve hesap silme için Deno tabanlı sunucusuz fonksiyonlar.
- Realtime — veri değişikliklerinin cihazlar arasında canlı senkronizasyonu.

Bu yaklaşımda istemci doğrudan veritabanına bağlanır; veri güvenliği ise uygulama kodu yerine veritabanı düzeyinde tanımlanan Row Level Security politikalarıyla sağlanır. Bu sayede yetkilendirme mantığı tek bir merkezde, veriye en yakın katmanda toplanmıştır (ayrıntı Bölüm 3).

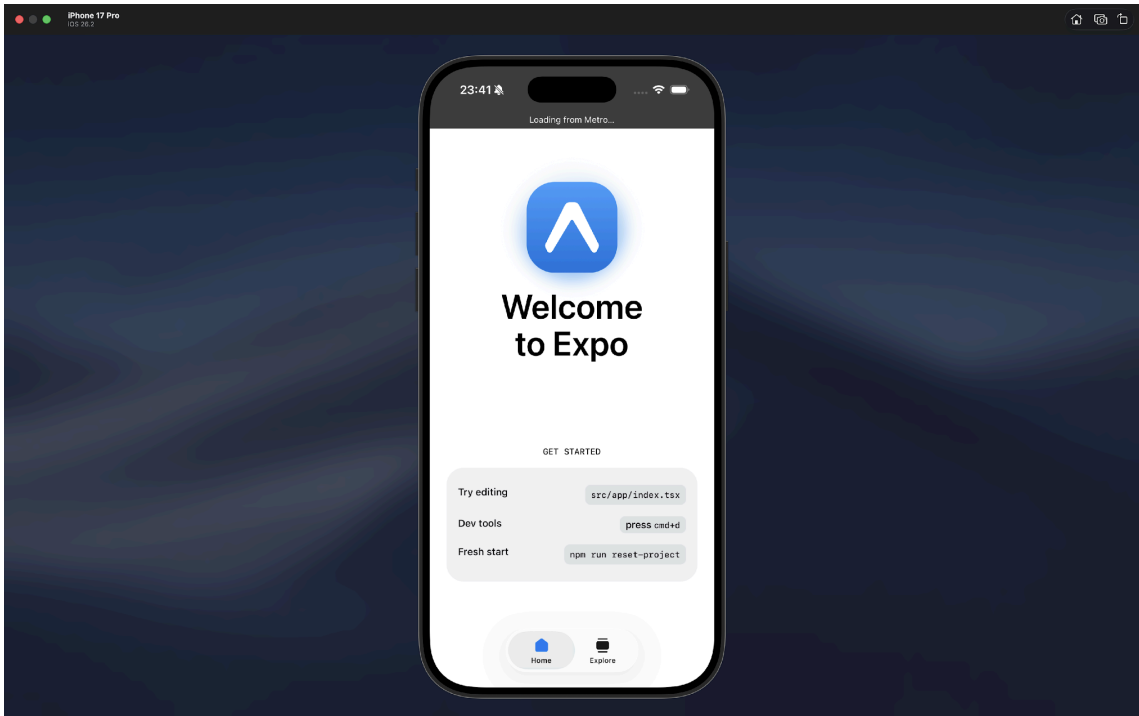
2.3. Katmanlı Uygulama Mimarisi

Uygulama kodu, tüm mantığın tek yerde toplandığı dağınık bir yapı yerine; ekran, bileşen, servis, durum ve yardımcı katmanlarının ayrıldığı modüler bir mimariyle düzenlenmiştir. Bu yapı, kod tekrarını azaltmakta ve her katmanın bağımsız olarak geliştirilip test edilmesine olanak vermektedir. Kaynak ağacı (src/) başlıca şu klasörlerden oluşur:

- app/ — Expo Router ekranları; (auth) giriş/kayıt ve (app) uygulama içi rotalar.
- components/ — yeniden kullanılabilir arayüz bileşenleri (buton, giriş alanı, kart, modal).
- services/ — Supabase ile konuşan veri erişim katmanı (gruplar, harcamalar, ödemeler, fiş).
- stores/ — Zustand tabanlı durum depoları (split-data, app-settings).
- hooks/ — türetilmiş veriler için özel React hook'ları (toplulaştırmalar, seçiciler).
- utils/ — saf yardımcı fonksiyonlar (borç hesabı, biçimlendirme, fiş ayırıştırma).
- lib/ — altyapı (Supabase istemcisi, MMKV deposu, ikon yeniden dışa aktarımları).
- types/ — uygulama genelinde paylaşılan TypeScript tip tanımları.

Servis katmanı, veritabanındaki snake_case sütunları uygulamadaki camelCase modellere dönüştüren bir eşleme (mapper) sorumluluğunu da üstlenir. Böylece arayüz ve durum katmanları, veritabanı şemasının ayrıntılarından yalıtılmış olur. MMKV gibi native modüller kullanıldığından uygulama Expo Go yerine geliştirme istemcisi (development build) ile çalıştırılmaktadır.

Durum yönetiminde, React 19 ile gelen katı anlık görüntü (snapshot) denetimi nedeniyle dikkat gerektiren noktalar olmuştur. Zustand seçicilerinin her render'da yeni bir nesne döndürmesi sonsuz render döngüsüne yol açabildiğinden; toplulaştırılmış veriler kök dilimler üzerinden useMemo ile sabitlenmiş ve kararlı seçiciler kullanılmıştır. Benzer biçimde, Tamagui'nin Sheet/portal bileşenlerinin birden çok kopyasının yarattığı bağlam çakışması, bağımlılık tekilleştirilmesiyle giderilmiştir. Bu tür ayrıntılar, modüler mimarinin yanı sıra kütüphane sürümleri arası uyumun da gözetildiğini göstermektedir.



Şekil 2. Expo geliştirme ortamı ve iOS Simulator üzerinde çalışan uygulama.

3. VERİTABANI TASARIMI VE GÜVENLİK

3.1. Tasarım İlkeleri

Veri modeli PostgreSQL üzerinde public şemasında kurulmuştur. Tüm kimlikler uuid tipindedir ve kullanıcı kimliği Supabase'in yönettiği auth.users tablosundaki kimlikle birebir örtüşür. Zaman alanları timestampz tipindedir. Parasal değerler, kayan nokta hatalarından kaçınmak için numeric(12,2) tipiyle saklanır; eşit bölüşümdeki yuvarlama farkı son kişiye atanarak toplamın korunması garanti edilir.

Veri kaybını önlemek için silme işlemleri kalıcı (hard delete) değil, yumuşak silme (soft delete) biçiminde tasarlanmıştır: ilgili tablolarda nullable bir deleted_at alanı bulunur ve uygulama sorguları deleted_at IS NULL koşuluyla filtrelenir. Böylece bir harcama veya grup silinse dahi geçmiş kayıtlar ve borç ilişkileri bozulmadan korunur.

3.2. İlişkisel Şema ve Tablolar

Şema; profil, arkadaşlık isteği, grup, üyelik, harcama, pay, ödeme ve etkinlik günlüğü tablolarından oluşur. Bir kullanıcının profili auth.users ile bire bir ilişkilidir; bir grup birden çok üyeye, bir harcama birden çok paya, bir grup birden çok harcama ve ödemeye sahiptir. Başlıca tablolar ve görevleri aşağıda özetlenmiştir.

Çizelge 2. SplitSnap veritabanı tabloları.

Tablo	Görevi
profiles	Kullanıcı profili; auth.users ile bire bir, benzersiz arkadaş kodu (user_invite_code)
friend_requests	Arkadaşlık istekleri; yalnızca arkadaş koduyla, RPC üzerinden
groups	Gruplar; sahip (owner_id), davet kodu, para birimi, soft delete
group_members	Grup üyelikleri; rol (admin/üye), katılım ve ayrılma zamanı
expenses	Harcamalar; tutar, ödeyen, bölüşüm tipi, fiş yolu, OCR önerileri
expense_shares	Harcamanın kullanıcı bazlı payları
settlements	Ödeme kayıtları (kimden kime, ne kadar)
activity_log (+ archive)	Etkinlik günlüğü ve arşivi (pg_cron ile taşıma)

Sorgu başarımını artırmak için sık erişilen alanlara kısmi (partial) indeksler eklenmiştir; örneğin yalnızca aktif (left_at IS NULL) üyeleri ya da silinmemiş (deleted_at IS NULL) harcamaları kapsayan indeksler, listeleme sorgularını hızlandırır. Birincil anahtar ve unique kısıtlamalar için PostgreSQL'in otomatik ürettiği indeksler ayrıca tanımlanmamıştır.

3.3. Otomatik Davranışlar: Tetikleyiciler ve Fonksiyonlar

Tutarlılığı uygulama koduna bırakmamak için bazı kurallar doğrudan veritabanında tanımlanmıştır. Yeni bir kullanıcı kaydolduğunda bir tetikleyici otomatik olarak profiles satırını oluşturur ve ad bilgisini kayıt meta verisinden alır. Yeni bir grup eklendiğinde davet kodu otomatik üretilir ve grup sahibi yönetici rolüyle ilk üye olarak eklenir. Grup davet kodu ile kullanıcı arkadaş kodunun çakışmaması, her iki tabloda da benzersizlik denetleyen ortak bir üretici fonksiyonla güvence altına alınmıştır.

Davet koduyla gruba katılma işlemi, istemciye doğrudan tablo erişimi vermek yerine SECURITY DEFINER yetkisine sahip bir RPC fonksiyonu (join_group_by_invite) üzerinden

yürütülür. Benzer şekilde harcama ve payları yazma işlemi, veri tutarlılığını korumak için tek bir işlemde (transaction) çalışan create_expense_with_shares ve update_expense_with_shares fonksiyonlarına taşınmıştır.

```
-- Davet koduyla gruba katılma (SECURITY DEFINER RPC)
create or replace function public.join_group_by_invite(code text)
returns uuid
language plpgsql security definer as $$
declare _group_id uuid;
begin
  select id into _group_id from public.groups
  where invite_code = code and deleted_at is null;
  -- ... üyelik ekleme / yeniden katılma mantığı ...
  return _group_id;
end; $$;
```

3.4. Satır Düzeyinde Güvenlik (Row Level Security)

Projede özel bir backend olmadığından, yetkilendirmenin tamamı veritabanı düzeyinde Row Level Security (RLS) politikalarıyla sağlanır. RLS her tabloda açıktır ve varsayılan davranış “erişim yok”tur; yalnızca açıkça yazılan politikaların izin verdiği satırlar görünür. Politikalarda oturum açan kullanıcının kimliği auth.uid() ile alınır. Yalnızca authenticated rolüne politika yazılmış, anonim erişime hiçbir kapı bırakılmamıştır.

Sık tekrar eden “bu kullanıcı bu grubun üyesi mi?” kontrolü, sonsuz özyineleme sorunlarını önlemek ve okunabilirliği artırmak için SECURITY DEFINER yardımcı fonksiyonlarına çıkarılmıştır: is_group_member (aktif üyeler) ve is_group_participant (ayrılmış üyeler dahil). Örneğin profil tablosunda bir kullanıcı yalnızca kendi satırını, ortak grupta olduğu ya da arkadaşlık ilişkisi bulunan kullanıcıların profillerini görebilir; böylece isim veya e-posta ile tüm kullanıcıların taranması engellenmiştir.

3.5. Güvenlik ve Tutarlılık Denetimi

Geliştirme sürecinde planlı bir güvenlik ve tutarlılık denetimi yapılmış; tespit edilen açıklar ve riskler giderilmiştir. Öne çıkan düzeltmeler şunlardır:

Çizelge 3. Güvenlik ve tutarlılık denetiminde giderilen başlıca bulgular.

Bulgu	Risk	Çözüm
Atomik olmayan yazma	Harcama eklenirken pay yazımı yarıda kalırsa veri tutarsızlığı	create/update_expense_with_shares RPC ile tek transaction
expense_shares RLS	Herhangi bir üye başkasının paylarını silebiliyordu	INSERT/UPDATE/DELETE politikalarına sahiplik denetimi eklendi
Soft-delete sızıntısı	Silinmiş gruplar listede görünebiliyordu	RLS ve istemci sorgularına deleted_at IS NULL filtresi
RLS özyinelemesi	group_members politikasının kendini sorgulaması sonsuz döngü	is_group_participant SECURITY DEFINER fonksiyonuna yönlendirme
Yuvarlama hatası	100 ₺ / 3 kişi toplamı 99,99 ₺ veriyordu	floor + kalan deseni ile toplamın korunması

brckit's Org FREE / splisnap / main **REDUCTION** / Connect

Expand sidebar by id, name, description, or ask AI

Sort 3 RLS policies Index Advisor Role postgres Insert

id	uid	name	text	description	text	owner_id	uid	invite_code	text	currency	text	created_at	timestampz	deleted_at	time
238a687-a99e-4bc4-8350-9f2844b66se		Viyana Gezisi		NULL		eb7875a-0e05-4518-97fe-d9d0..		9F355980		TRY		2026-04-11 22:19:54.222914+00		NULL	
70c27a0-6abn-43a7-857d-3f2a8178916		Gibns Gezisi		NULL		eb7875a-0e05-4518-97fe-d9d0..		BB8fC3A		TRY		2026-04-11 22:22:30.38369+00		NULL	

Page 1 of 1 100 rows 2 records

Data Definition

Şekil 3. Supabase üzerinde grup ve üyelik verilerinin tablo görünümü.

4. KIMLIK DOĞRULAMA, GRUP VE ÜYELİK YÖNETİMİ

4.1. Kayıt, Giriş ve Oturum Yönetimi

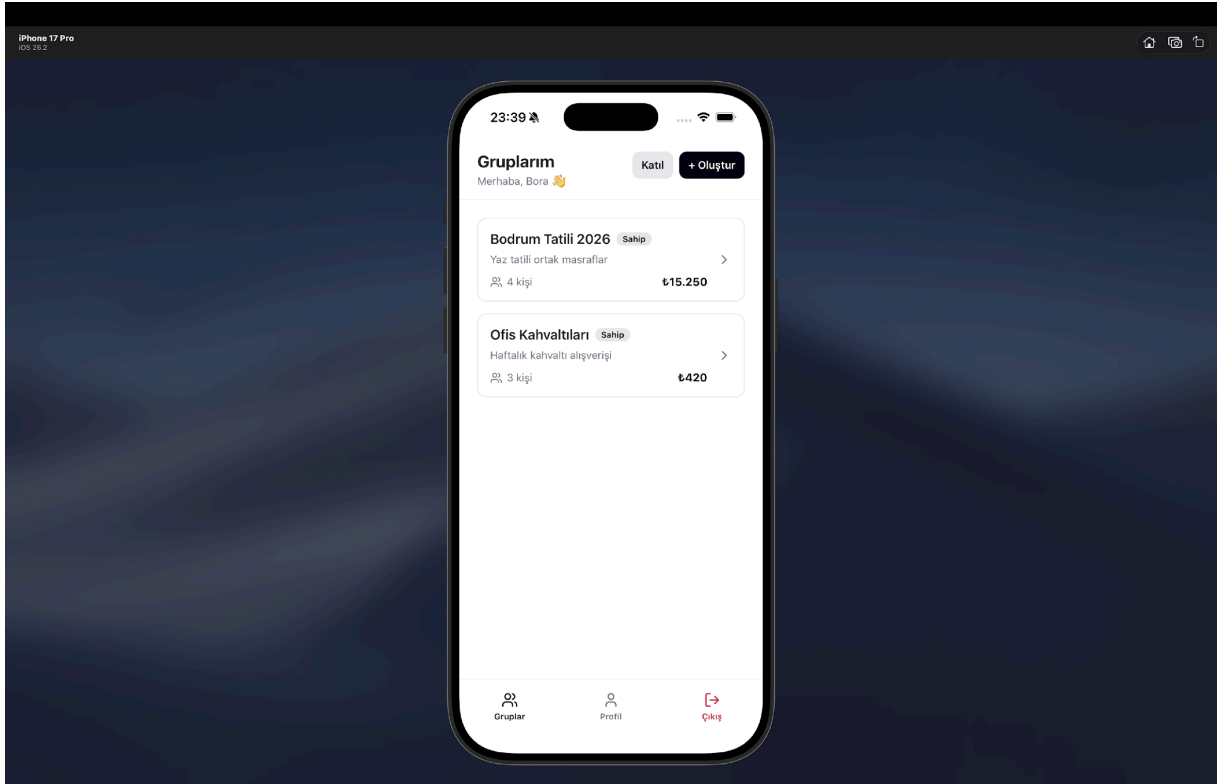
Kullanıcı kimlik doğrulaması Supabase Auth ile gerçekleştirilir. Kayıt ekranında ad, e-posta ve şifre alınır; ad bilgisi kayıt meta verisine yazılır ve veritabanı tetikleyicisi tarafından profil satırına aktarılır. Giriş ve kayıt formlarında istemci tarafı doğrulama yapılır: e-posta biçimi, en az altı karakterlik şifre ve en az iki karakterlik ad gibi kurallar, hatalı girişleri sunucuya gitmeden satır içi uyarılarla engeller.

Oturum durumu, bir bağlam (auth-context) üzerinden uygulama genelinde yönetilir. Supabase'in onAuthStateChange olayı dinlenerek kullanıcı giriş yaptığında uygulama ekranlarına, çıkış yaptığında giriş akışına yönlendirme otomatik yapılır. Çıkışta arka plan senkronizasyon süreçleri durdurularak gereksiz ağ trafiği ve sızıntı önlenir. Geliştirme sürecinde geçici olarak kullanılan giriş atlama (dev login bypass) mekanizması, yalnızca gerçek Supabase Auth akışı kalacak şekilde kaldırılmıştır.

4.2. Grup Oluşturma ve Üyelik

Kullanıcı yeni bir grup oluşturduğunda, veritabanı tarafında grup kaydı eklenir, benzersiz bir davet kodu üretilir ve kullanıcı yönetici rolüyle ilk üye olarak atanır. Grup listesi ekranı; her grubun adını, üye sayısını ve toplam harcama tutarını kartlar hâlinde gösterir. Liste ilk açılışta tüm grupların harcama toplamalarını toplu bir sorguyla yükler; böylece kullanıcı gruba girmeden önce dahi doğru toplamaları görür.

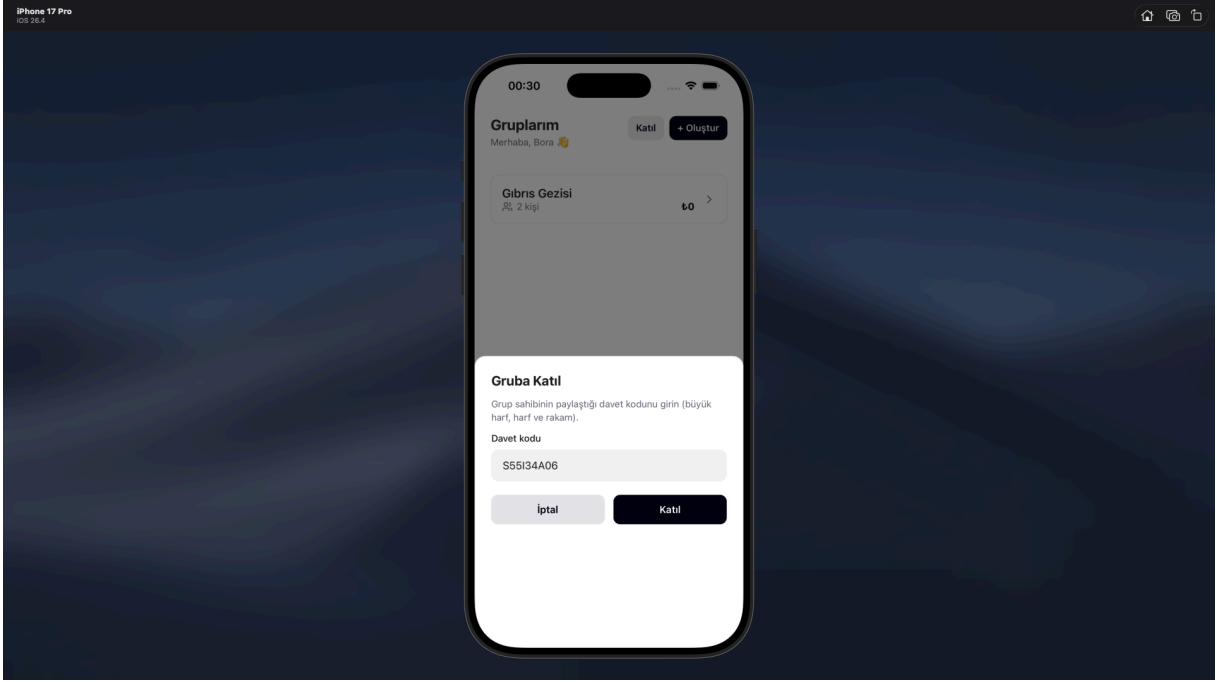
Grup ve harcama verileri Supabase Realtime aboneliğiyle canlı olarak senkronize edilir; bir cihazda yapılan değişiklik diğer üyelerin ekranına kısa sürede yansır. Uygulama ön plana geldiğinde de veriler yeniden çekilerek güncellik korunur.



Şekil 4. Grup listesi ekranı: grup kartları, üye sayısı ve toplam tutar.

4.3. Davet Kodu ve Gruba Katılma

Gruba katılma, isim veya e-posta ile serbest arama yerine davet kodu üzerinden tasarlanmıştır. Bu, gizlilik açısından bilinçli bir karardır: hiçbir kullanıcı diğerlerini listeleyerek tarayamaz. Katılma işlemi, davet kodunu doğrulayan ve üyelik satırını ekleyen SECURITY DEFINER yetkili bir RPC fonksiyonuyla yürütülür. Daha önce gruptan ayrılmış bir kullanıcı aynı kodla yeniden katılabilir.



Şekil 5. Davet kodu ile gruba katılma akışı.

4.4. Tıklanabilir Davet Bağlantıları (Universal Link)

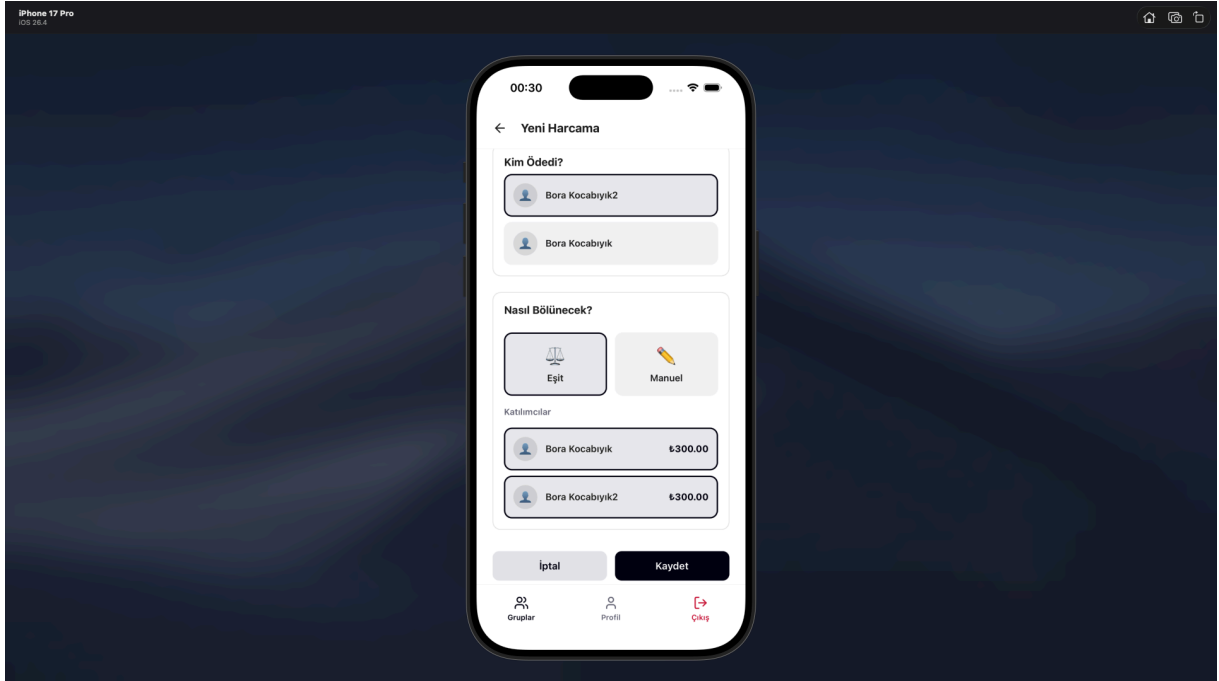
Davet deneyimini kolaylaştırmak için davet kodları, App Store yayını aşamasında tıklanabilir evrensel bağlantılara (universal link) dönüştürülmüştür. Bir kullanıcı `splitsnap.borak.dev/invite/<KOD>` biçimindeki bağlantıyı paylaştığında, bağlantıya dokunan kişi uygulamayı yüklüyse doğrudan ilgili gruba katılma akışına yönlendirilir; yüklü değilse web sitesindeki yedek sayfa açılır ve App Store'a yönlendirilir. Giriş yapılmamışsa davet kodu yerel olarak (MMKV) saklanır ve giriş sonrası otomatik katılım tamamlanır. Bu akışın altyapısı Bölüm 8'de ayrıntılandırılmıştır.

5. HARCAMA YÖNETİMİ VE BORÇ HESAPLAMA

5.1. Harcama Ekleme, Düzenleme ve Silme

Harcama yönetimi uygulamanın çekirdek işlevidir. Kullanıcı bir grupta yeni harcama eklerken başlık, tutar, tarih, harcamayı ödeyen kişi ve harcamaya katılan üyeleri belirtir. Tutar girişinde yalnızca sayısal değere izin verilir; başlık uzunluğu ve tutar üst sınırı gibi doğrulamalar form üzerinde anlık olarak uygulanır. Tarih alanı, metin girişi yerine iOS'un yerel tarih seçici bileşeniyle doldurulur.

Harcamalar Supabase'de expenses tablosunda, kullanıcı bazlı paylar ise expense_shares tablosunda saklanır. Ekleme ve güncelleme işlemleri, harcama ile paylarının her zaman birlikte ve tutarlı yazılmasını sağlamak için tek bir veritabanı işlemi içinde çalışan RPC fonksiyonları üzerinden yapılır. Silme işlemi kalıcı değildir; harcama soft-delete ile işaretlenir, böylece geçmiş borç ilişkileri korunur. Silme sonrası kullanıcıya kısa bir bildirim (toast) gösterilir.



Şekil 6. Harcama ekleme ekranı: ödeyen seçimi, katılımcılar ve eşit bölüşüm.

5.2. Eşit ve Manuel Bölüşüm

Bir harcama iki biçimde paylaştırılabilir. Eşit bölüşümde tutar, seçilen katılımcı sayısına bölünür ve her katılımcının payı anında hesaplanarak gösterilir. Tam bölünemeyen tutarlarda kuruluş farkı son kişiye eklenerek payların toplamının harcama tutarına eşit kalması garanti edilir. Manuel bölüşümde ise her kişi için ayrı bir tutar girilir; arayüz, kalan tutarı ("Kalan: ₺...") satır içinde göstererek ve kişi başına üst sınır uygulayarak hatalı girişleri engeller.

5.3. Borç-Alacak Hesabı ve Ödeşme Optimizasyonu

Uygulamanın temel mantıksal bileşeni, grup içindeki net borç-alacak durumunu çıkaran hesaplama katmanıdır. Her üye için, ödediği toplam tutar ile üzerine düşen payların toplamı arasındaki fark net bakiyeyi verir: pozitif bakiye alacaklı, negatif bakiye borçlu olduğunu

gösterir. Hesaba yalnızca harcamalar değil, kaydedilmiş ödemeler de dâhil edilir; böylece bir ödeme yapıldığında bakiyeler güncellenir.

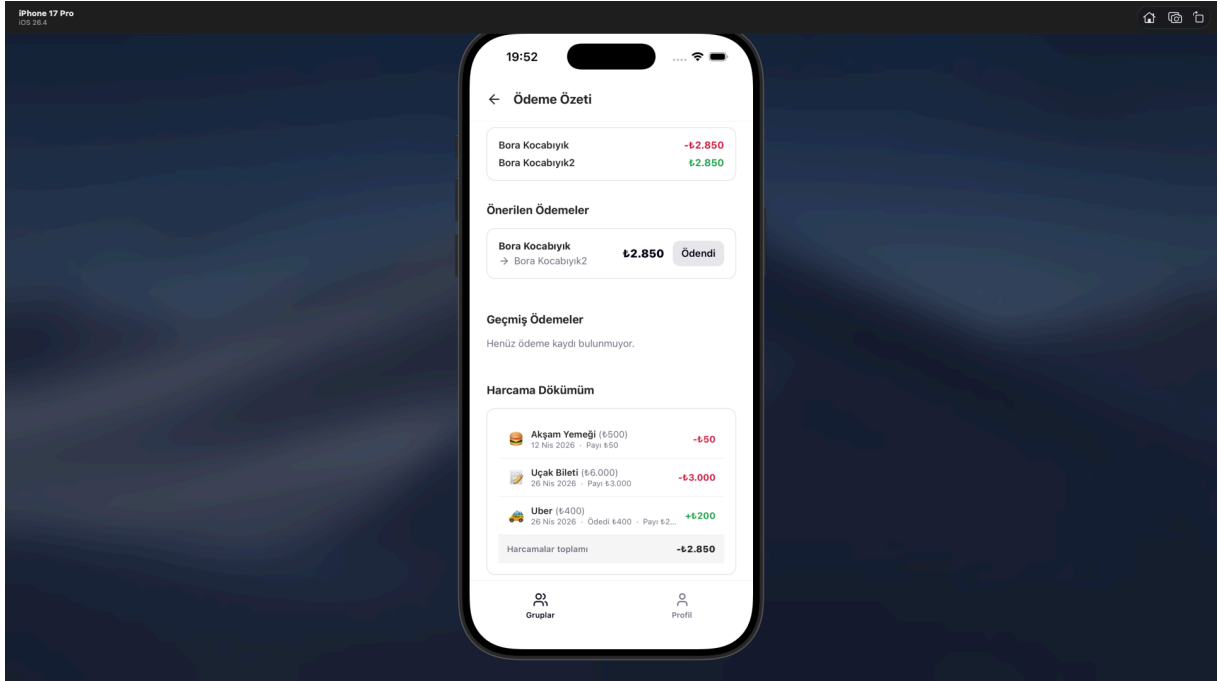
Net bakiyeler çıkarıldıktan sonra, herkesin en az sayıda işlemle ödemesini sağlayan bir eşleştirme algoritması çalışır. Borçlular ve alacaklılar bakiye büyüklüğüne göre açgözlü (greedy) biçimde eşleştirilir; en büyük borç en büyük alacakla kapatılacak şekilde önerilen ödemeler listesi üretilir. Bu yaklaşım, grup içindeki ödeme sayısını azaltarak “kim kime ne kadar ödemeli?” sorusuna sade bir yanıt verir.

Yöntem somut bir örnekle açıklanabilir. Üç kişilik (A, B, C) bir grupta A 120 ₺’lik, B ise 60 ₺’lik birer harcama yapmış ve her ikisi de üç kişi arasında eşit bölüşülmüş olsun. Toplam 180 ₺’nin kişi başı payı 60 ₺’dir. Buna göre A 120 ödeyip 60 pay aldığından +60 (alacaklı), B 60 ödeyip 60 pay aldığından 0 (eşit), C ise hiç ödemeyip 60 pay aldığından -60 (borçlu) net bakiyeye sahip olur. Algoritma, tek borçlu C’yi tek alacaklı A ile eşleştirir ve sonuç tek bir öneriye iner: C, A’ya 60 ₺ öder. Üyeler arası karmaşık ara ödemeler yerine en az sayıda işlem önerilmesinin değeri, grup ve harcama sayısı arttıkça daha da belirginleşir.

Çizelge 4. Örnek borç-alacak hesabı (öneri: C → A 60 ₺).

Üye	Ödediği	Düşen Pay	Net Bakiye	Durum
A	120 ₺	60 ₺	+60 ₺	Alacaklı
B	60 ₺	60 ₺	0 ₺	Eşit
C	0 ₺	60 ₺	-60 ₺	Borçlu

Ödeme ekranı; kullanıcının net bakiyesini, tüm üyelerin bakiyelerini, önerilen ödemeleri ve geçmiş ödemeleri bir arada sunar. Ayrıca “Harcama Dökümüm” bölümü, kullanıcının dâhil olduğu her harcama için ödediği tutar, üzerine düşen pay ve net katkısı ayrıntılı olarak listeler. Bir ödeme gerçekleştiğinde kullanıcı “Ödendi” işlemiyle bunu kaydeder; ilgili ödeme settlements tablosuna yazılır ve bakiyelere yansır.



Şekil 7. Ödeme ekranı: net bakiye, üye bakiyeleri ve önerilen ödemeler.

5.4. Kullanıcı Deneyimi İyileştirmeleri

Çekirdek akışın üzerine pek çok deneyim iyileştirmesi eklenmiştir: grup listesi, grup detayı ve ödeme ekranlarında aşağı çekerek yenileme (pull-to-refresh); harcama başlığından kategori tahmin ederek otomatik emoji atama; renkli baş harf tabanlı avatar sistemi; üye listesini daraltıp genişletme; ve “Alacaklısınız / Borçlusunuz / Eşitsiniz” gibi durum açıklamaları. Emoji ataması, platform genelindeki kullanım istatistiklerini bir RPC ile çekip yerel olarak önbelleğe alan dinamik bir haritayla zenginleştirilmiştir.

6. YAPAY ZEKÂ DESTEKLİ FİŞ TARAMA

Uygulamanın en ayırt edici özelliği, bir fiş fotoğrafından harcama bilgilerini otomatik olarak çıkaran fiş tarama akışıdır. Amaç, kullanıcının harcama eklerken işletme adı, tarih ve toplam tutar gibi alanları elle yazma yükünü ortadan kaldırmaktır. Bu bölümde, özelliğin mimari kararları, uçtan uca işleyişi, gizlilik ve maliyet boyutu ile değerlendirilen alternatifler ayrıntılı olarak ele alınmaktadır.

6.1. Mimari Karar: Hibrit Model

Fiş okuma için iki uçta iki yaklaşım vardı. Birincisi, fiş metnini saf yerel kurallarla (regex) ayrıştırmak; ikincisi, görseli doğrudan bir bulut servisine göndererek uçtan uca işletmek. Saf yerel ayrıştırma, fişlerin biçim çeşitliliği nedeniyle çok sayıda kırılğan kural yazmayı gerektirir ve hataya açıktır. Görseli buluta göndermek ise gizlilik açısından sakıncalıdır ve maliyet doğurur.

Bu nedenle bir hibrit model benimsenmiştir: metin tanıma (OCR) cihaz üzerinde, ücretsiz ve gizliliği koruyacak biçimde yapılır; yalnızca elde edilen ham metin, yapılandırılmış alanlara dönüştürülmek üzere bir dil modeline gönderilir. Böylece görsel hiçbir zaman cihazdan çıkmaz, yüksek hacimli kural yazma ihtiyacı tamamen ortadan kalkar ve dil modelinin esnekliğinden yararlanır.

6.2. Uçtan Uca İşleyiş

Fiş tarama akışı, kullanıcı bir fiş fotoğrafı çektiğinde ya da galeriden seçtiğinde başlar ve aşağıdaki adımlardan geçer:

- Görsel hazırlama: Fotoğraf, yükleme ve işleme öncesinde en fazla 1600 piksele küçültülüp JPEG olarak sıkıştırılır (expo-image-manipulator).
- Cihaz içi OCR: Apple Vision tabanlı metin tanıma (expo-text-extractor) ile fişin ham metni çıkarılır. Bu adım çevrimdışı çalışır ve görsel cihazdan ayrılmaz.
- Yapılandırma (LLM): Ham metin, Supabase üzerinde çalışan parse-receipt adlı Edge Function'a gönderilir; fonksiyon, OpenAI gpt-4o-mini modelini katı bir JSON şemasıyla çağırarak metinden işletme adı, tarih, toplam tutar ve para birimi alanlarını çıkarır.
- Otomatik doldurma: Dönen alanlar harcama formuna yazılır. Kullanıcının daha önce doldurduğu alanların üzerine yazılmaz; yalnızca boş alanlar doldurulur.
- Bölüşüme bağlanma: Okunan toplam tutar form alanına yazıldığında, mevcut eşit bölüşüm hesabı kendiliğinden devreye girer; ayrı bir adım gerekmez. Kullanıcı dilerse manuel bölüşüme geçebilir, OCR ile çakışma olmaz.

```
# İstemci tarafı akış (özet)
uri      = await pickReceiptImage()      # kamera / galeri
resized  = await manipulate(uri, maxW=1600) # küçült + JPEG q0.7
rawText  = await extractTextFromImage(resized) # Apple Vision (cihazda)
result   = await parseReceipt(rawText)      # Edge Function -> gpt-4o-mini
# result = { merchantName, date, total, currency }
autofillExpenseForm(result)              # yalnızca boş alanları doldur
```

6.3. Dayanıklılık: Yerel Sezgisel Yedek

Edge Function'a erişilemediği durumlarda (anahtar tanımsız, çevrimdışı veya hata) akış kesintiye uğramaz; istemci tarafındaki yerel bir sezgisel ayrıştırıcı devreye girer. Bu yedek; "TOPLAM/ÖDENECEK" gibi anahtar sözcükleri, gg.aa.yyyy biçimli tarihleri ve para birimi

sembollerini tarayarak temel alanları çıkarmaya çalışır. Böylece özellik, bulut servisine tam bağımlı olmadan en azından kısmi sonuç üretebilir.

6.4. Para Birimi Tespiti ve Yabancı Döviz Uyarısı

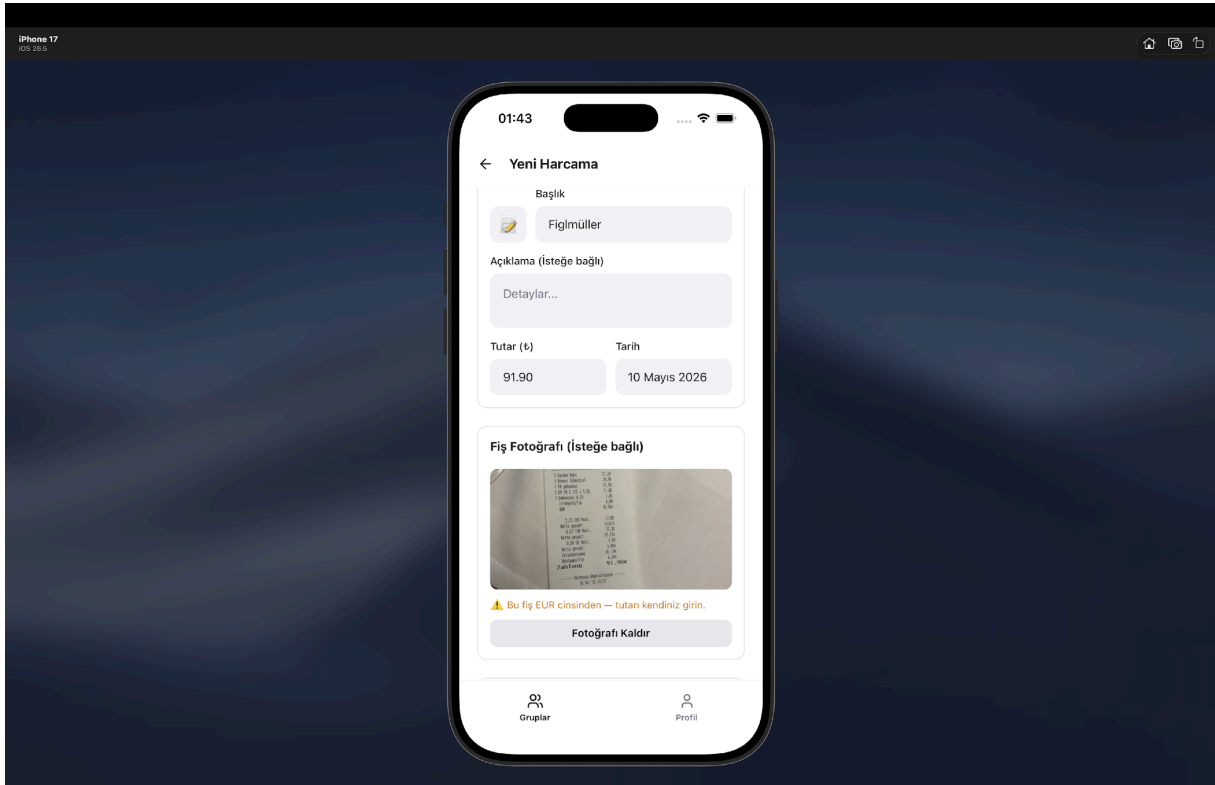
Fişten okunan para birimi alanı (TRY, EUR, USD, GBP) hem Edge Function hem de yerel sezgisel tarafından döndürülür. Uygulama grup bazında tek para birimiyle (varsayılan TRY) çalıştığından, Türk Lirası dışında bir fiş algılandığında tutarın otomatik doldurulması atlanır ve kullanıcıya turuncu bir uyarı (⚠️) gösterilir. Bu, yanlış para biriminden kaynaklanabilecek hatalı bölüşümleri önleyen bilinçli bir korumadır.

6.5. Gizlilik ve Maliyet

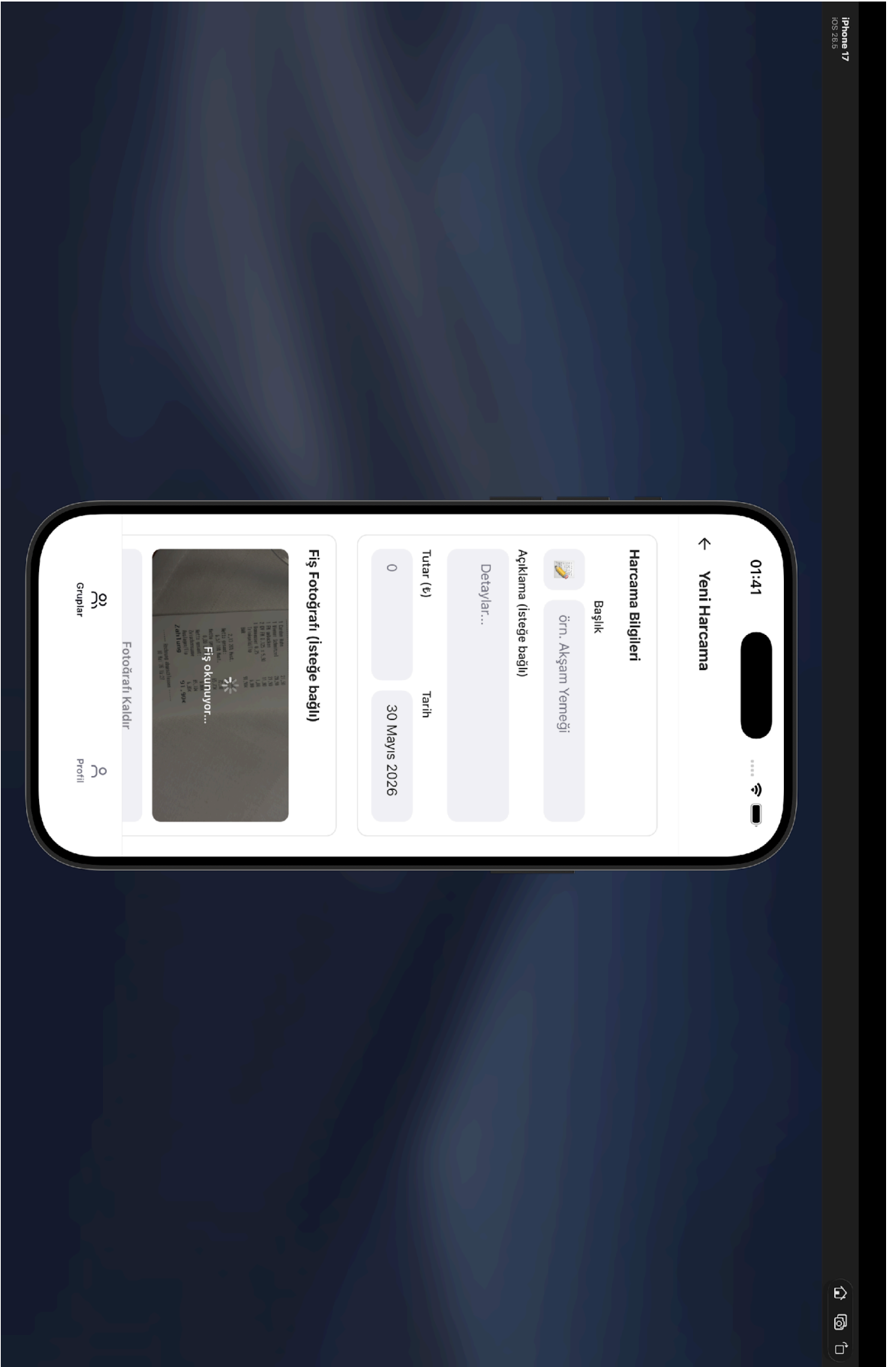
Hibrit tasarımın doğrudan sonucu güçlü bir gizlilik duruşudur: fiş görselinin kendisi hiçbir zaman buluta gönderilmez; yalnızca anonimleştirilmiş ham metin dil modeline ulaşır ve bu metinde kişisel tanımlayıcı bilgi bulunmaz. Maliyet açısından da yöntem oldukça verimlidir: tipik bir fiş için yaklaşık 541 jeton tüketilir ve gpt-4o-mini fiyatlandırmasında bu, fiş başına ihmal edilebilir bir maliyete karşılık gelir.

6.6. Değerlendirilen Alternatif: GLM-OCR

Daha güçlü, görseli doğrudan işleyen modeller de değerlendirilmiştir. Bunlardan biri, OmniDocBench v1.5 ölçütünde lider konumdaki Zhipu AI'nin GLM-OCR modelidir. Ancak bu modelin kullanımı, fiş görselinin buluta gönderilmesini gerektirdiğinden gizlilik açısından bir gerilemeye yol açacaktı. Basit fişler için mevcut hibrit yöntem zaten yeterli doğruluk ve düşük maliyet sunduğundan, gizliliği koruyan mevcut mimari korunmuştur.



Şekil 8. Fiş okuma sonucu: işletme adı, tarih ve tutarın otomatik doldurulması.



Şekil 9. OCR sürerken fiş görseli üzerindeki yükleniyor göstergesi.

7. YEREL VERİ SAKLAMA, PERFORMANS VE OPTİMİZASYON

7.1. MMKV ile Yerel Veri Saklama

Uygulamanın ana veri kaynağı Supabase olmakla birlikte; açılış hızını ve kullanıcı deneyimini iyileştirmek için veri katmanı yerel olarak da saklanır. Bunun için, senkron ve yüksek hızlı okuma sunan MMKV deposu kullanılmıştır. Zustand durum deposu, persist ara katmanı ile MMKV'ye bağlanmış; yalnızca veri dilimleri (gruplar, üyeler, harcamalar, paylar, ödemeler) kalıcılaştırılırken fonksiyonlar dışarıda bırakılmıştır. Sonuç olarak uygulama kapatılıp yeniden açıldığında önceki oturumun verileri anında görüntülenir; ağ yanıtı beklenmez.

Tema tercihi de (sistem/açık/koyu) aynı yerel saklama mekanizmasıyla kalıcı hâle getirilmiştir. Açılış akışında, splash ekranı yerel verinin geri yüklenmesi ve oturum denetiminin tamamlanmasını bekler; grup senkronizasyonu ise arka planda, açılış bloklamadan yürütülür. Böylece açılış süresi ağ gecikmesinden bağımsız hâle gelmiştir.

7.2. Başarım Optimizasyonları

Dönem içinde yürütülen başarım denetiminde, ölçülebilir iyileştirmeler sağlayan bir dizi optimizasyon uygulanmıştır. Bunların başlıcaları aşağıdaki çizelgede özetlenmiştir.

Çizelge 5. Başlıca başarım optimizasyonları ve etkileri.

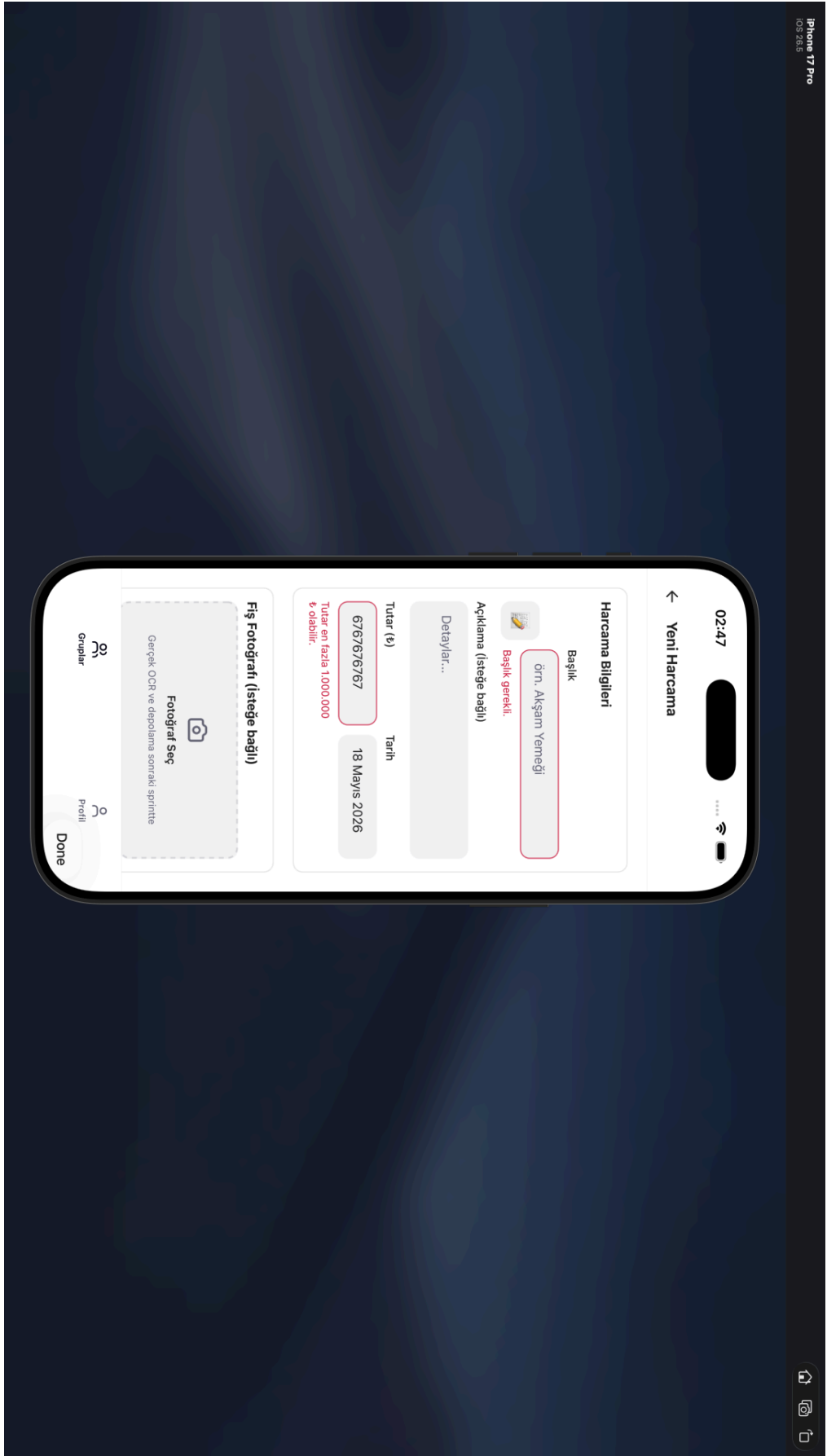
Optimizasyon	Yöntem	Etki
Realtime debounce	Ardışık canlı olayları 300 ms ile tek isteğe indirme	API çağrılarında ~5-10 kat azalma
Paralel veri çekme	Sıralı sorguları Promise.all ile paralelleştirme	~%33 gecikme azalması
Store erişimi	Array.find ($O(n^2)$) yerine Set tabanlı arama ($O(n)$)	Büyük listelerde daha hızlı güncelleme
Atomik yazma	Harcama + payları tek transaction'da yazma	Veri tutarsızlığı riskinin ortadan kalkması
Önceden hesaplama	Grup toplam/üye sayısını useMemo Map ile $O(1)$ erişim	Liste yeniden render maliyetinin düşmesi

7.3. Paket Boyutu ve Derleme İyileştirmeleri

Üretim derlemesinin boyutunu küçültmek için ikon kütüphanesinde ağaç sarsma (tree-shaking) uygulanmıştır: Metro paketleyicisine özel bir çözümleyici eklenip ikonlar tek bir merkezden yeniden dışa aktarılarak, paketlenen ikon sayısı 1703'ten 19'a indirilmiş; JavaScript bytecode boyutu yaklaşık %24 azalmıştır (5,0 MB → 3,8 MB). Ayrıca uygulama ikonu görsel kalitesi korunarak 988 KB'tan 116 KB'a sıkıştırılmış; dokümantasyon, tasarım ve web sitesi dosyaları .easignore ile derleme yüklemelerinden dışlanmış. Kod kalitesi tarafında 24 lint hatası sıfıra indirilerek temiz bir derleme tabanı elde edilmiştir.

7.4. Form Doğrulama ve Profil Yönetimi

Kullanıcı girdileri hem deneyim hem güvenlik açısından doğrulanır: e-posta biçimi, şifre uzunluğu, ad uzunluğu, harcama tutarı üst sınırı ve başlık uzunluğu gibi kurallar form üzerinde anlık olarak denetlenir. Profil tarafında ise ad düzenleme ve şifre değiştirme ekranları eklenmiştir; şifre değişiminde mevcut şifre doğrulanır, yeni şifre için uzunluk, eşleşme ve aynı-şifre engeli denetlenir.



Şekil 10. Form doğrulama: hatalı girişlerin satır içi uyarılarla engellenmesi.

8. APP STORE YAYINI VE DAĞITIM

Dönem başındaki tanıtım raporunda App Store yayını açıkça kapsam dışı (gelecek sürüm) olarak belirtilmişti. Buna karşın proje, gerçek bir ürün deneyimi yaşamak amacıyla uçtan uca yayın sürecinden geçirilmiş ve SplitSnap Apple App Store'da yayımlanmıştır. Bu bölüm; derleme ve gönderim hattını, App Store Connect ayarlarını, Apple'ın zorunlu kıldığı gizlilik ve uyumluluk gereksinimlerini, hesap silme akışını ve davet bağlantıları için kurulan web altyapısını ele almaktadır.

8.1. EAS ile Derleme ve Gönderim Hattı

Üretim derlemeleri, yerel makinede değil EAS (Expo Application Services) bulutunda alınır. eas.json dosyasında development, preview ve production profilleri tanımlıdır. Üretim profili autoIncrement ayarıyla her derlemede yapı numarasını otomatik artırır; pazarlama sürümü (1.0.0) ise elle yönetilir. Süreç iki komutla işler: bulutta derleme (eas build) ve App Store Connect'e gönderim (eas submit). Günlük geliştirme ise simülatör veya USB ile bağlı fiziksel cihaz üzerinde, bu hattan bağımsız yürütülür.

Çizelge 6. EAS derleme profilleri.

Profil	Kullanım
development	Cihaz kaydı + geliştirme istemcisi (dev client)
preview	Kayıtlı cihazlara dahili test dağıtımı
production	App Store derlemesi (autoIncrement) → eas submit

The screenshot displays the App Store Connect interface for a project named 'SplitSnap'. The 'TestFlight' tab is selected, showing a list of iOS builds. Below this, the 'Version 1.0.0' details are visible, including a table of build statistics.

Build	Status	Groups	Invites	Installs	Sessions	Crashes	Feedback
9	Ready to Submit Expires in 30 days	10	1	-	-	-	-
6	Ready to Submit Expires in 30 days	10	1	2	-	6	2
3	Ready to Submit Expires in 30 days	10	1	1	-	4	2
2	Ready to Submit Expires in 30 days	10	1	-	-	-	-

Şekil 11. App Store Connect / TestFlight üzerinde alınan üretim derlemeleri.

8.2. App Store Connect Ayarları

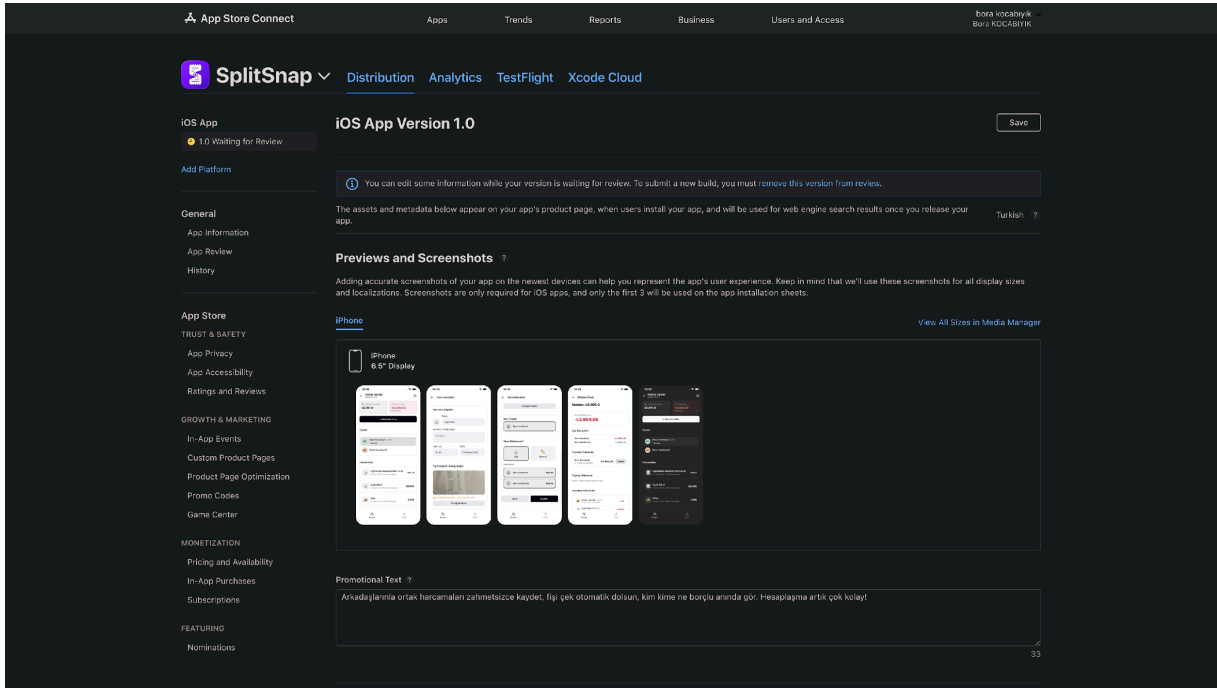
Uygulama, App Store Connect üzerinde Finans kategorisinde, “Grup harcamalarını fişle böl” alt başlığıyla yapılandırılmıştır. Yaş sınırı, içerik soruları doğrultusunda 4+ olarak hesaplanmıştır. Gizlilik politikası ve destek bağlantısı projenin web sitesine yönlendirilmiş;

inceleme ekibi için, oturum açma gerektiren bir uygulama olduğundan, mevcut bir gruba dâhil edilmiş bir demo hesabı ve OCR testi için yönerge sağlanmıştır.

Apple'ın istediği Gizlilik Bilgi Etiketi (Privacy Nutrition Label) kapsamında toplanan veri tipleri ve amaçları beyan edilmiştir. Toplanan tüm veriler yalnızca uygulama işlevselliği amaçlıdır; hiçbir izleme (tracking) yapılmaz, reklam veya veri toplayıcı (data broker) bağımlılığı yoktur. OpenAI'ye yalnızca fiş metni gönderilir; görsel veya doğrudan kişisel bilgi paylaşılmaz.

Çizelge 7. Gizlilik Bilgi Etiketi — toplanan veri tipleri (tümü uygulama işlevselliği amaçlı).

Veri Tipi	Kimliğe Bağlı	İzleme
Ad	Evet	Hayır
E-posta	Evet	Hayır
Kullanıcı kimliği	Evet	Hayır
Finansal bilgi (harcama tutarları)	Evet	Hayır
Fotoğraf (fiş görselleri)	Evet	Hayır



Şekil 12. App Store Connect — uygulama bilgileri ve meta verileri.

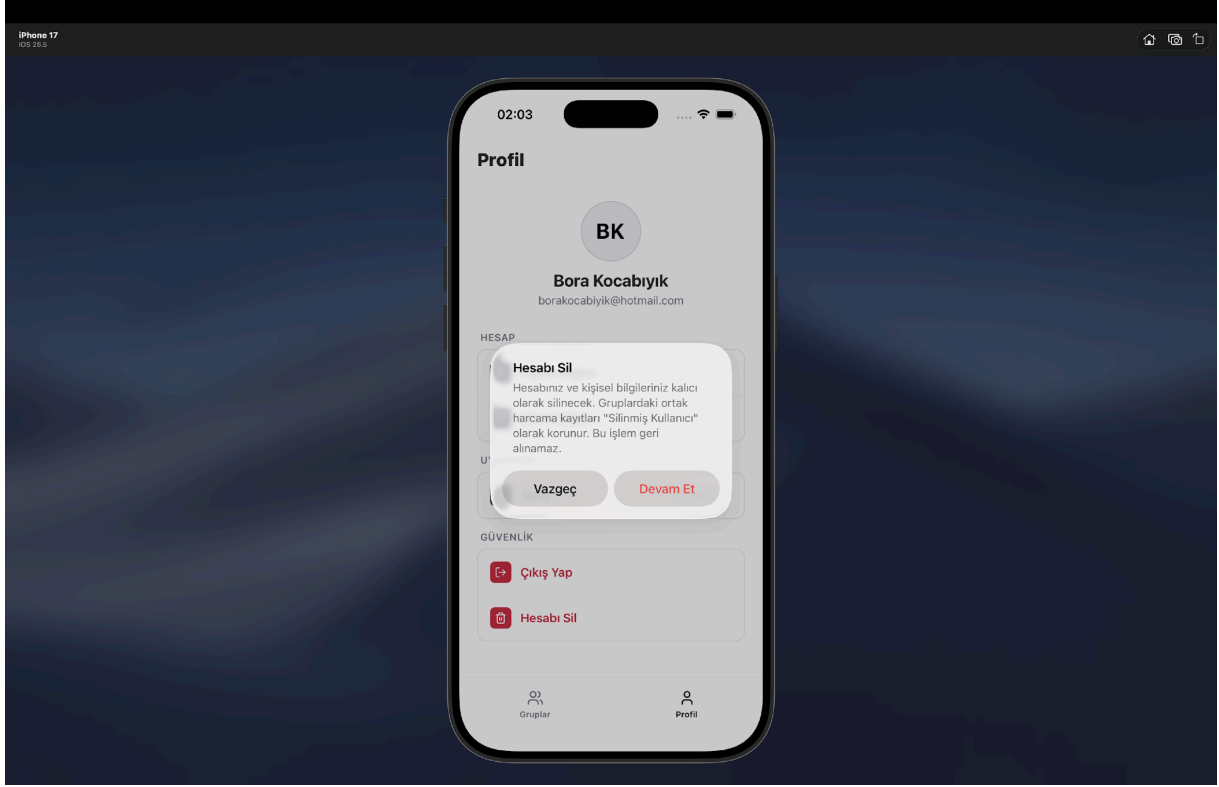
8.3. Gizlilik Manifestosu ve Uyumluluk

Apple'ın 2024'te zorunlu kıldığı gizlilik manifestosu kapsamında, uygulamanın kullandığı API kategorileri ve gerekçe kodları beyan edilmiştir: kullanıcı ayarlarını saklamak için UserDefaults, kullanıcı eylemlerine yanıt için dosya zaman damgası ve içerik yazımı öncesi disk alanı kontrolü. Şifreleme uyumluluğu tarafında uygulama yalnızca standart HTTPS kullandığından muaf kapsamdadır ve ek bir ihracat bildirimi gerektirmez. Davet bağlantıları ve iOS otomatik doldurma için Associated Domains yetkisi (applinks ve webcredentials) etkinleştirilmiştir.

8.4. Hesap Silme Akışı

Apple'ın 5.1.1(v) kuralı uyarınca, hesap oluşturan uygulamaların uygulama içinden hesap silme sunması gerekir. SplitSnap'te bu, Profil → Hesabı Sil yolundan erişilen ve

delete-account adlı Edge Function ile yürütülen bir akıştır. Kalıcı (hard) silme yerine anonimleştirme tercih edilmiştir: kullanıcının profili “Silinmiş Kullanıcı” olarak anonimleştirilir, kimliği erişilemez kılınır. Bunun nedeni, harcama kayıtlarının ödeyen kullanıcıya zorunlu bir yabancı anahtarla (RESTRICT, NOT NULL) bağlı olması ve kalıcı silmenin ortak harcama geçmişini bozacak olmasıdır. Böylece kullanıcı verisi erişilemez hâle gelirken grup geçmişinin bütünlüğü korunur.



Şekil 13. Uygulama içi hesap silme akışı (Apple 5.1.1(v)).

8.5. Web Sitesi ve Davet Bağlantısı Altyapısı

Davet bağlantılarının, gizlilik politikasının ve evrensel bağlantı doğrulama dosyasının (AASA) yayımlanması için splitsnap.borak.dev adresinde bir web sitesi kurulmuş ve Cloudflare Pages üzerinde yayımlanmıştır. Site; ana sayfa, gizlilik politikası, AASA dosyası ve davet bağlantıları için yedek bir sayfa içerir. Bir davet bağlantısına uygulama yüklü olmayan bir cihazdan dokunulduğunda bu yedek sayfa açılır ve kullanıcı App Store'a yönlendirilir; uygulama yüklüyse evrensel bağlantı doğrudan katılım akışını tetikler.

8.6. Yayın Aşamasında Çözülen Sorunlar

Üretim derlemesine geçişte, geliştirme ortamında görülmeyen bir açılış çökmesi tespit edilmiştir: ortam değişkenlerinin üretim derlemesine taşınmaması nedeniyle Supabase istemcisi geçersiz bir yapılandırma ile başlatılıyor ve uygulama açılışta çöküyordu. Sorun, ortam değişkenlerinin EAS yapılandırmasına eklenmesi ve istemci başlatmanın savunmacı (geçersiz yapılandırmada güvenli yedeğe düşen) hâle getirilmesiyle çözülmüş; düzeltme yeni bir derlemede TestFlight üzerinde doğrulanmıştır. Bu deneyim, üretim ve geliştirme ortamları arasındaki farkların gerçek cihazda test edilmesinin önemini göstermiştir.



Gizlilik Politikası

Son güncelleme: 29 Mayıs 2026

SplitSnap ("Uygulama"), grup içi harcamaları kaydetmenizi, fiş fotoğraflarından otomatik bilgi çıkarmanızı ve borç-alacak hesabını paylaşmanızı sağlayan bir mobil uygulamadır. Bu politika, Uygulama'yı kullanırken hangi verilerin işlendiğini ve haklarınızı açıklar.

1. İşlenen Veriler

Veri	Amaç
Ad ve e-posta adresi	Hesap oluşturma, giriş ve oturum yönetimi
Grup, harcama ve ödeme kayıtları	Uygulamanın temel işlevi (harcama bölüşümü ve hesaplama)
Fiş fotoğrafları	Harcama kaydına ek olarak saklama ve OCR ile otomatik doldurma
Fişten çıkarılan metin	İşleme adı, tarih ve tutarı yapılandırılması (örneğin OpenAI maddesine bakınız)
Cihaz üzeri teknik veriler	Oturumun ve uygulama tercihlerinin cihazda saklanması

2. Verilerin Saklandığı Yer

Hesap ve uygulama verileri, altyapı sağlayıcımız **Supabase** üzerinde (PostgreSQL veritabanı ve dosya depolama) güvenli şekilde tutulur. Fiş görselleri özel (private) bir depolama alanında saklanır ve yalnızca ilgili grubun üyeleri tarafından süreli iznizle bağlantılara erişilebilir.

3. Üçüncü Taraf: OCR / Yapay Zeka

Fiş okuma özelliğinde, görüntüden metne dönüştürme işlemi **cihazınızda** (Apple Vision) yapılır, fiş görseli **buluta gönderilmez**. Yalnızca elde edilen **ham metin**, işleme adı/tarihi/tutar alanlarını yapılandırmak üzere **OpenAI** (gpt-4o-mini) servisine iletilir. Bu metin OpenAI tarafından model eğitiliminde kullanılmaz ve yanıt alındıktan sonra kalıcı olarak saklanmaz.

Şekil 14. splitsnap.borak.dev — Cloudflare Pages üzerinde yayımlanan tanıtım sitesi.

9. SONUÇ VE DEĞERLENDİRME

9.1. Plana Göre Gerçekleşme

Dönem başındaki tanıtım raporunda hedeflenen çalışan ve sunulabilir iOS uygulaması tamamlanmıştır. Planlanan tüm çekirdek özellikler; kayıt ve giriş, oturum yönetimi, grup oluşturma ve davet koduyla katılma, üye görüntüleme, harcama ekleme/düzenleme/silme, ödeyen ve katılımcı seçimi, eşit ve manuel bölüşüm, borç-alacak hesabı, ödeme özeti, fiş ekleme ve OCR ile alan doldurma, yerel veri saklama gerçekleştirilmiştir. Bunun da ötesinde, planda açıkça gelecek sürüm/kapsam dışı olarak işaretlenen App Store yayını da tamamlanmış ve uygulama gerçek bir ürün olarak yayımlanmıştır. Bu yönüyle proje, başlangıç hedefinin ötesine geçmiştir.

9.2. Teknik Kazanımlar

Proje, modern bir mobil uygulamanın uçtan uca geliştirilmesinde rol alan katmanların tümünde uygulamalı deneyim sağlamıştır. React Native/Expo ile native özelliklere erişen bir mimari kurulmuş; TypeScript ile tip güvenli bir model katmanı oluşturulmuştur. Backend tarafında, özel sunucu yazmadan Supabase üzerinde ilişkisel bir şema tasarlanmış ve güvenlik, satır düzeyinde RLS politikalarıyla veriye en yakın katmanda sağlanmıştır. Borç-alacak hesabı ve en az işlemle ödeme optimizasyonu ile uygulama, veri girişinin ötesinde gerçek bir hesaplama sistemi hâline gelmiştir. Fiş tarama özelliğinde benimsenen hibrit model ise gizlilik, maliyet ve doğruluk arasında dengeli bir mühendislik kararını temsil etmektedir.

9.3. Karşılaşılan Zorluklar

Geliştirme boyunca çeşitli teknik zorluklarla karşılaşmış ve çözülmüştür: RLS politikalarının kendini sorgulamasından doğan sonsuz özyineleme, SECURITY DEFINER yardımcı fonksiyonlarıyla aşılmış; harcama ve paylarının ayrı yazılmasından kaynaklanan tutarsızlık riski, atomik RPC fonksiyonlarıyla giderilmiş; üretim derlemesine özgü açılış çökmesi, ortam değişkenlerinin yapılandırılması ve savunmacı başlatma ile çözülmüştür. Bu deneyimler, üretim ve geliştirme ortamlarının ayrı ayrı doğrulanmasının ve güvenliğin tasarımın başından itibaren ele alınmasının önemini göstermiştir.

9.4. Gelecek Geliştirmeler

Mevcut sürüm sağlam bir temel oluşturmakla birlikte, ürünü büyütecek bir dizi geliştirme gelecek sürümlere bırakılmıştır:

- Çoklu dil desteği: uygulama metinlerinin İngilizce/Türkçe seçeneğiyle yerelleştirilmesi.
- Çoklu ödeyen desteği: bir harcamanın birden fazla kişi tarafından karşılanabilmesi.
- Arkadaşlık sistemi: arkadaş koduyla istek gönderme akışının arayüze tam yansıtılması.
- Çoklu para birimi: harcama veya grup bazında para birimi ve kur dönüşümü.
- Android desteği ve fişten ürün bazlı (kalem kalem) ayrıştırma.

Sonuç olarak SplitSnap, ders kapsamında belirlenen hedefleri karşılayan; kimlik doğrulama, ilişkisel veri, güvenlik, hesaplama ve yapay zekâ bileşenlerini bir araya getiren ve gerçek kullanıcılara açık biçimde yayımlanmış, bütünlüklü bir mobil yazılım projesidir.

KAYNAKÇA

Çatı, Dil ve İstemci Kütüphaneleri

- [1] Expo. Expo Documentation. <https://docs.expo.dev/>
- [2] Expo. Expo Router — File-based routing for React Native. <https://docs.expo.dev/router/introduction/>
- [3] Expo. EAS Build & Submit. <https://docs.expo.dev/eas/>
- [4] Meta. React Native Documentation. <https://reactnative.dev/>
- [5] Tamagui. Tamagui — UI kit for React Native. <https://tamagui.dev/>
- [6] Poimandres. Zustand — State management for React. <https://zustand-demo.pmnd.rs/>
- [7] Marc Rousavy. react-native-mmkv. <https://github.com/mrousavy/react-native-mmkv>

Backend, Veritabanı ve Yapay Zekâ

- [8] Supabase. Supabase Documentation — Auth, Database, Storage, Edge Functions, Realtime. <https://supabase.com/docs>
- [9] Supabase. Row Level Security (RLS) Guide. <https://supabase.com/docs/guides/auth/row-level-security>
- [10] Supabase. Database Functions ve pg_cron. https://supabase.com/docs/guides/database/extensions/pg_cron
- [11] PostgreSQL Global Development Group. PostgreSQL 17 Documentation. <https://www.postgresql.org/docs/>
- [12] OpenAI. API Reference — Structured Outputs (JSON Schema). <https://platform.openai.com/docs/guides/structured-outputs>
- [13] OpenDataLab. OmniDocBench — Document parsing benchmark. <https://github.com/opendatalab/OmniDocBench>

Platform ve Dağıtım

- [14] Apple. App Store Review Guidelines. <https://developer.apple.com/app-store/review/guidelines/>
- [15] Apple. Privacy Manifest Files. https://developer.apple.com/documentation/bundleresources/privacy_manifest_files
- [16] Apple. Supporting Associated Domains (Universal Links). <https://developer.apple.com/documentation/xcode/supporting-associated-domains>
- [17] Apple. Human Interface Guidelines. <https://developer.apple.com/design/human-interface-guidelines>
- [18] Cloudflare. Cloudflare Pages Documentation. <https://developers.cloudflare.com/pages/>

Proje Çıktıları

- [19] SplitSnap. Proje deposu (GitHub). <https://github.com/brckfrc/splitsnap>
- [20] SplitSnap. App Store sayfası. <https://apps.apple.com/app/id6774394620>
- [21] SplitSnap. Tanıtım sitesi. <https://splitsnap.borak.dev>